

Introduction to C++

Introduction to Computer Graphics - Exercise 01

Pirmin Pfeifer

C++ Learning Resources:

C++ Learning Resources:

- ▶ C++20 Fundamentals (Online Course)

C++ Learning Resources:

- ▶ C++20 Fundamentals (Online Course)
- ▶ Effective Modern C++ (Book)
- ▶ Modern C++ for Absolute Beginners (Book)

C++ Learning Resources:

- ▶ C++20 Fundamentals (Online Course)
- ▶ Effective Modern C++ (Book)
- ▶ Modern C++ for Absolute Beginners (Book)
- ▶ W3 Schools C++ Introduction

C++ Learning Resources:

- ▶ C++20 Fundamentals (Online Course)
- ▶ Effective Modern C++ (Book)
- ▶ Modern C++ for Absolute Beginners (Book)
- ▶ W3 Schools C++ Introduction
- ▶ C# vs C++ Comparison

C++ Learning Resources:

- ▶ C++20 Fundamentals (Online Course)
- ▶ Effective Modern C++ (Book)
- ▶ Modern C++ for Absolute Beginners (Book)
- ▶ W3 Schools C++ Introduction
- ▶ C# vs C++ Comparison
- ▶ C++ Documentation

What is C++

C++ is a low level programming language which features:



What is C++

C++ is a low level programming language which features:



► memory management

What is C++

C++ is a low level programming language which features:



- ▶ memory management
 - ▶ object-orientation
-

What is C++

C++ is a low level programming language which features:



- ▶ memory management
- ▶ object-orientation
- ▶ compilation on all platforms^a

^aSpecial limitation may occur on micro controllers

What is C++

C++ is a low level programming language which features:



- ▶ memory management
- ▶ object-orientation
- ▶ compilation on all platforms^a
- ▶ Many ways to shoot yourself into the foot!

^aSpecial limitation may occur on micro controllers

Hello World!

```
1  #include <iostream>
2  int main()
3  {
4      std::cout << "Hello_World.";
5  }
```

Hello World!

```
1  #include <iostream>
2  int main()
3  {
4      std::cout << "Hello_World.";
5  }
```

1. Include *iostream* from the standard library (it contains `std::cout`)

Hello World!

```
1  #include <iostream>
2  int main()
3  {
4      std::cout << "Hello World.";
5  }
```

1. Include *iostream* from the standard library (it contains `std::cout`)
2. *main* function is the entry point and return an exit code as *int*

```
1  #include <iostream>
2  int main()
3  {
4      std::cout << "Hello_World.";
5  }
```

1. Include *iostream* from the standard library (it contains `std::cout`)
2. *main* function is the entry point and return an exit code as *int*
3. Open function block
4. The `<<` operator inserts our string literal into that output stream
5. Close function block


```
1  int main()
2  {
3      // Declaration of variable
4      // contains unknown state
5      bool b;
6      // Declaration + =Initialization
7      bool b = true;
8      // Declaration + {}Initialization
9      bool b{ true };
10 }
```

```
1  int main()
2  {
3      bool b = true; // truth state
4      char c = 'a';  // ASCII character
5      int x = 123;    // Int (unkown size)
6      int y = -256;   // but mostly 32bit
7
8      int32_t a = -125; // 32bit int
9      uint32_t a = 234; // 32bit unsigned int
10     int16_t a = 5123; // 16bit int
11     uint8_t a = 128;  // 8bit unsigned int
12
13     double = 3.14; // floating point 64bit
14     float  = 3.14; // floating point 32bit
15 }
```

```
1  int main()  
2  {  
3      unsigned int y = 0xFFFFFFFF;  
4  
5      const float f = 0;  
6      f = 1; // will not compile  
7  }
```

sizeof returns the size of the given type in bytes:

```
1  int main()  
2  {  
3      size_t a = sizeof(char);    // 1  
4      size_t a = sizeof(int32_t); // 4  
5      size_t a = sizeof(size_t);  // 8  
6  }
```

Arithmetic Operators

```
1  +  //  addition
2  -  //  subtraction
3  *  //  multiplication
4  /  //  division
5  %  //  modulo
6
7  += //  compound addition
8  -= //  compound subtraction
9  *= //  compound multiplication
10 /= //  compound division
11 %= //  compound modulo
```

```
1  #include <array>
2  int main()
3  {
4      int arr[5] = { 10, 20, 30, 40, 50 };
5
6      // even better:
7      std::array<int, 5> stdArr= {
8          10, 20, 30, 40, 50
9      };
10
11     arr[0] = 100;
12     int i = stdArr[3];
13 }
```

```
1  #include <vector>
2  int main()
3  {
4      std::vector<int> vec = {};
5      vec.emplace_back(5);
6      vec.emplace_back(7);
7      vec.insert(
8          vec.end(),
9          vec.begin(),
10         vec.end()
11     );
12     int a = vec[3];
13     int* p = vec.data();
14     vec.clear();
15 }
```

```
1  int main()
2  {
3      int* p;
4
5      int x = 123;
6      p = &x;
7      // memory address of variable p
8      size_t adr = (size_t) p;
9      // change p to point to nothing
10     p = nullptr;
11 }
```



```
1  int main()
2  {
3      // create new int on the heap
4      int* i = new int(50);
5      *i = *i / 5;
6      // copy onto stack
7      int l = *i;
8      // release memory from the heap
9      delete i;
10 }
```

```
1  int main()
2  {
3      // create new int on the heap
4      int* i = new int(50);
5      *i = *i / 5;
6      // copy onto stack
7      int l = *i;
8      // release memory from the heap
9      delete i;
10 }
```

Forgetting to release memory, which was acquired on the heap, results in a memory leak.

```
1  int main()
2  {
3      // create new int on the heap
4      int* i = new int(50);
5      *i = *i / 5;
6      // copy onto stack
7      int l = *i;
8      // release memory from the heap
9      delete i;
10 }
```

Forgetting to release memory, which was acquired on the heap, results in a memory leak.

Trying to access a released *ptr*, produces an error.

```
1  int main()
2  {
3      // create new unique int on heap
4      std::unique_ptr<int> p =
5          std::make_unique<int>(9);
6      // get "normal" Pointer
7      int* pInt = p.get();
8      int x = *p;
9      // clear ptr
10     p.reset();
11     p = nullptr;
12 }
```

```
1  int main()
2  {
3      int x = 123;
4      // y is a Reference/Alias for x
5      int& y = x;
6      x = 456;
7      // both x and y now hold the value 456
8      y = 789;
9      // both x and y now hold the value 789
10 }
```

```
1  #include <string>
2  int main()
3  {
4      std::string s = "Hello_";
5      s += "World";
6      char c = '!';
7      s += c;
8
9      char c1 = s[2];           // 'l'
10
11     std::string s2 = s.substr(3,2); // "lo"
12     // evaluates to true
13     bool b = s == "Hello_World!";
14 }
```

```
1  #include <string>
2  int main()
3  {
4      auto c = 'a';    // char type
5      auto x = 123;    // int type
6      auto d = 123.456 / 789.10; // double
7
8      auto& y = x; // y is of int& type
9
10     // z is of const int type
11     const auto z = 123;
12 }
```

```
1  int square(int i);
2  int main()
3  {
4      int a = 10;
5      int b = square(a);
6  }
7
8  int square(int i) {
9      return i * i;
10 }
```



```
1  // int i is referenced instead of copied
2  int square(int& i)
3  {
4      i = i * i;
5      return i;
6  }
7
8  int main()
9  {
10     int a = 10;
11     // a and b are equal 100
12     int b = square(a);
13 }
```

```
1  #include <iostream>
2  // function without return type
3  void printLine(std::string_view str) {
4      std::cout << str << std::endl;
5  }
6
7  int main()
8  {
9      print("Hello_World!");
10 }
```

Types - void*

```
1  int main()
2  {
3      int arr1[3] = { 0,1,2 };
4      int arr2[3] = { };
5      // conversion into void* is implicit
6      void* src = arr1;
7      void* dest = arr2;
8      // takes two void pointers
9      // and copys bytes from
10     // src to dest
11     std::memcpy(
12         dest,
13         src,
14         sizeof(int) * 3);
15 }
```

```
1 // std is the standard namespace
2 namespace mycode {
3     void print(){}
4 }
5 // we resolve namespaces with ::
6 void print2(){
7     mycode::print();
8 }
```

```
1  class MyClass{};
2  struct MyStruct{};
3  int main()
4  {
5      MyClass o;
6      MyStruct s;
7  }
```

```
1  class MyClass{};  
2  struct MyStruct{};  
3  int main()  
4  {  
5      MyClass o;  
6      MyStruct s;  
7  }
```

The difference between *class* and *struct* is default accessibility.

```
1  class MyClass{};  
2  struct MyStruct{};  
3  int main()  
4  {  
5      MyClass o;  
6      MyStruct s;  
7  }
```

The difference between *class* and *struct* is default accessibility.

- *struct* is *public* by default

```
1  class MyClass{};
2  struct MyStruct{};
3  int main()
4  {
5      MyClass o;
6      MyStruct s;
7  }
```

The difference between *class* and *struct* is default accessibility.

- ▶ *struct* is *public* by default
- ▶ *class* is *private* by default

Classes - Member Fields

```
1  class MyClass{
2  public:
3      int a;
4      bool b;
5      double d;
6  };
7  int main()
8  {
9      MyClass o;
10     o.a = 5;
11     o.b = false;
12     o.d = 3.14;
13 }
```

```
1  class MyClass{
2  public:
3      int a;
4      int getA(){
5          return a;
6      }
7  };
8  int main()
9  {
10     MyClass o;
11     o.a = 7;
12     int a = o.getA();
13 }
```

```
1  class MyClass{
2  public:
3      int  getA(){
4          return a;
5      }
6  private:
7      int  a = 8;
8  };
9  int  main()
10 {
11     MyClass  o;
12     int  a = o.getA();
13     // dose not compile:
14     a = o.a;
15 }
```

```
1  class MyClass{
2  public:
3      MyClass() = default;
4      MyClass(int i);
5  private:
6      int a = 8;
7  };
8  MyClass::MyClass(int i){ a = i;}
9  int main()
10 {
11     MyClass o1;
12     int a = o1.getA(); // a == 8
13     auto o2 = MyClass(2);
14     a = o2.getA();      // a == 2
15 }
```

```
1  // Class.h
2  class MyClass {
3  public:
4      double getPI();
5      double getPISquared();
6  }
7  // Class.cpp
8  #include "Class.h"
9  double MyClass::getPI(){ return 3.14; }
10 double MyClass::getPISquared(){
11     return 9.86
12 }
```

Any Questions?

